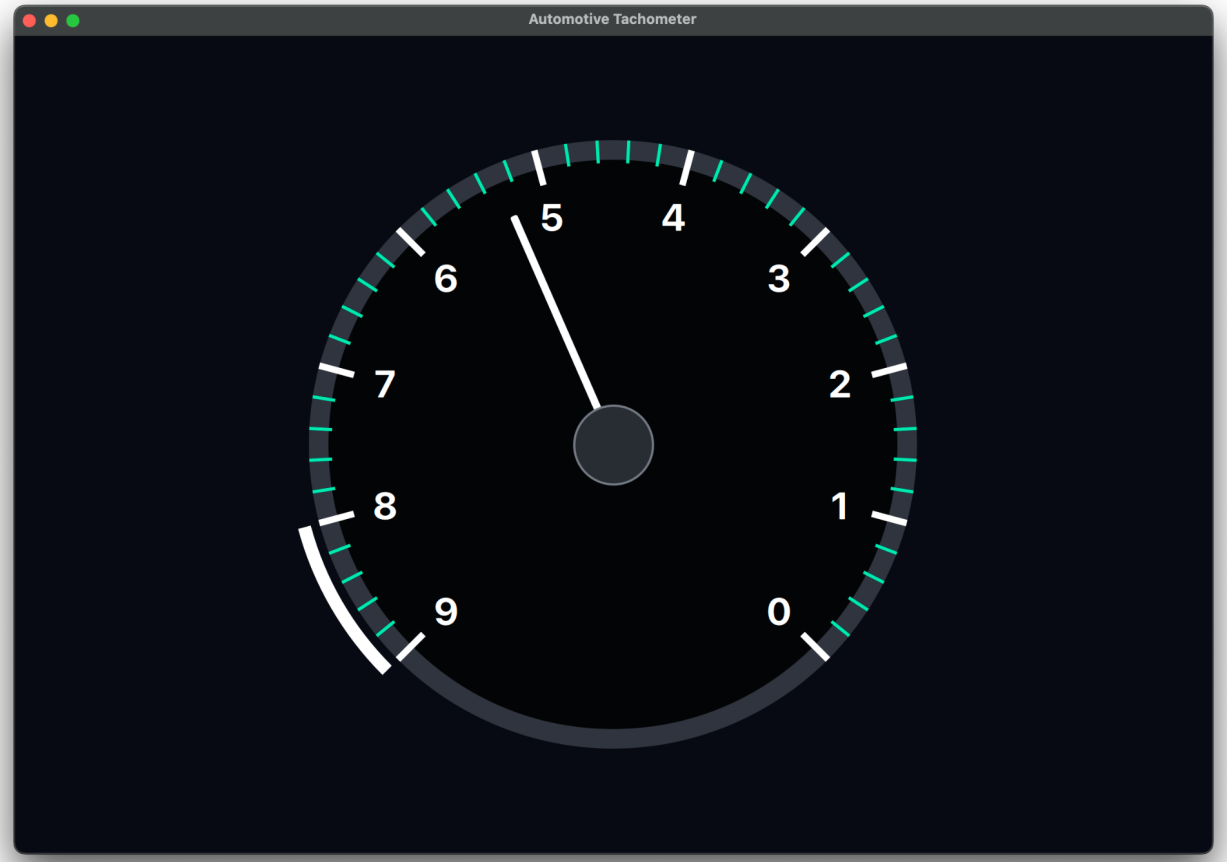
AUTOMOTIVE TACHOMETER

Technical Integration Proposal | Qt 6 / C++ / QML

Document status: Prototype assessment and integration planning | Version 1.0 | 12 November 2026

Purpose: Present the existing desktop RPM gauge prototype, clarify its limitations, and define the engineering activities needed before integration with a vehicle or an automotive display platform.



Important: This screenshot is evidence of a rendered desktop user interface, not evidence of vehicle connectivity, functional-safety compliance, real-time performance, or production readiness.

1. Executive assessment

The current implementation demonstrates an RPM dial rendered with Qt Quick/QML, a moving needle, and a C++ simulation component designed to expose RPM through Qt properties and signals. It is suitable for a design review and software demonstration, subject to build and runtime verification. It is not ready for direct vehicle deployment.

The latest screenshot also shows a visual discrepancy: the numbering runs from 0 at lower right toward 9 at lower left, while the supplied target design places 0 at lower left and 9 at lower right. This must be corrected and visually approved.

2. Scope and evidence

Area	Current status / evidence
UI	Desktop screenshot shows dial, numerals, graduations, needle and high-RPM arc.
RPM simulation	C++ EngineSimulator code proposed; verify that it is present, compiled, and connected in the submitted build.
Input	Keyboard acceleration control proposed; must be tested on the delivered binary.
ECU/CAN	Not implemented or demonstrated.
Target hardware	Not selected or validated.
Safety/certification	No automotive safety or compliance claim.

3. Reference software architecture

Proposed data flow: Vehicle ECU / approved gateway -> vehicle network (e.g., CAN) -> hardware CAN controller / OS driver -> transport adapter -> validated signal decoder -> RPM data model -> Qt QML tachometer.

Keep the data source independent from the presentation. The QML dial must consume a stable RPM interface; a simulated provider and a vehicle provider should be interchangeable without rewriting the gauge.

Suggested components

- Tachometer.qml: drawing, tick layout, needle animation, scaling and visual warning state.
- RpmModel (QObject): read-only rpm, dataValid, signalAgeMs, sourceState and change notifications.
- SimulationProvider: deterministic test input for desktop demos.
- CanRpmProvider: platform-specific frame acquisition and decode; must use OEM-approved signal definitions.
- SignalValidator: range checks, timeout handling, invalid-value handling and explicit quality state.
- ApplicationController: source selection, diagnostics, initialization and shutdown.

4. Vehicle data interface - information required from OEM

No CAN identifier, byte layout, scaling factor, or message frequency can be assumed from this prototype. The vehicle integrator must provide an approved DBC/ARXML or equivalent signal specification and gateway/interface documentation.

Required input	Reason
Vehicle/platform, ECU and display hardware	Determine electrical, OS, display and interface constraints.
CAN / CAN FD / other bus topology	Select physical interface, controller and driver.

Required input	Reason
RPM signal definition (ID, start bit, length, endian, factor, offset, unit)	Decode engine speed correctly; values are vehicle-specific.
Update period, timeout, validity and diagnostic rules	Define freshness and fault behavior.
Ignition / power-state requirements	Startup, sleep, shutdown and brownout behavior.
Approved integration access and security rules	Avoid unauthorized interaction with vehicle systems.

5. Data handling and display behavior

- Display valid engine speed in RPM, mapped to the approved gauge range; do not silently clamp invalid raw data and present it as trustworthy.
- If frames are missing, stale, malformed, or explicitly invalid, show a defined unavailable/fault indication; do not keep displaying the last value indefinitely as live.
- Use the signal specification to determine engine-off zero, idle behavior, peak range, and redline. The current 0-9000 scale is illustrative only.
- Use monotonic timestamps for freshness checks; separate signal update rate from render frame rate.
- Smooth only the visual needle, with a bounded lag approved by the integrator. Animation must not conceal abrupt faults or invalid data.
- Treat the tachometer as a display consumer: it must not issue ECU control commands.

6. Platform and build integration

A macOS Qt Quick desktop demonstration is not equivalent to an automotive target build. The company must select the production SoC/MCU, operating system (for example embedded Linux or an approved RTOS), graphics stack, screen resolution, input model, and deployment packaging.

- Confirm supported Qt version, licensing obligations, cross-compilation toolchain, Qt Quick rendering backend and available GPU acceleration.
- Confirm CAN driver availability and whether Qt SerialBus/SocketCAN is supported on the selected platform; otherwise implement a dedicated approved transport adapter.
- Define startup time, memory/CPU/GPU budgets, frame-rate target, thermal limits, screen luminance and day/night presentation.
- Provide reproducible builds, pinned dependencies, source control revision, binary checksum, deployment and rollback instructions.
- Avoid GUI thread blocking: receive and decode network frames asynchronously, publish validated state to the UI thread.

7. Functional safety, security and regulatory review

A vehicle instrument display can be safety-relevant. The OEM or designated safety engineering team must perform hazard analysis and assign safety requirements, including any applicability of ISO 26262. Security engineering should assess applicable ISO/SAE 21434 and UNECE R155/R156 obligations according to vehicle market, architecture and scope. The prototype does not claim certification under any of these standards.

- Define safe behavior for loss of communication, invalid RPM, frozen rendering, restart and power interruption.
- Evaluate whether an independent warning or fallback display path is required by the vehicle safety concept.
- Use approved gateways, access controls and update mechanisms; never test on a moving vehicle or live control network without OEM authorization and a controlled test plan.
- Check human-machine-interface legibility, distraction and telltale requirements with the responsible compliance team.

8. Verification and acceptance plan

Phase	Required evidence / exit criteria
A. Desktop design review	Approved visual comparison, correct number direction, needle mapping at 0/idle/mid/max, scalable layout.
B. Unit testing	Range, scaling, invalid input, timestamp and timeout tests with repeatable data.
C. Integration bench	Approved replayed/simulated frames through target adapter; signal trace correlated with UI.
D. Target performance	Measured latency, dropped frames, memory, CPU/GPU, startup and long-duration stability.
E. Fault injection	Communication loss, malformed frames, stale data, engine-off and restart scenarios.
F. Vehicle validation	OEM-supervised testing under an approved safety plan; documented sign-off.

9. Recommended deliverables

- Source repository containing CMake, Qt/QML, C++ and README with exact build instructions.
- Reproducible desktop demo and short screen recording of controlled RPM input.
- Visual reference comparison and UI specification (colors, geometry, states, scaling).
- Interface contract describing RPM value, unit, validity, timestamps and errors.
- Platform adapter design and approved signal specification (provided by company).
- Test plan, test logs, measured performance and known-issues register.
- Release notes, dependency inventory, licensing notes and handover checklist.

10. Implementation phases and ownership

Phase	Owner / action
1. Visual alignment	UI developer: fix dial direction, refine metal ring, warning band and needle.
2. Software hardening	UI/C++ developer: separate model/providers, deterministic tests, errors and logs.
3. Vehicle interface definition	OEM: provide signal spec, hardware and access rules.
4. Bench integration	Embedded team + developer: implement transport, decode and replay tests.
5. Qualification	OEM: safety, security, performance and vehicle acceptance.

11. Open decisions for the company

- Which vehicle model, instrument cluster hardware, operating system and Qt deployment model are targeted?
- Is RPM available through CAN, CAN FD, a gateway service or another interface?
- What is the authoritative signal definition and fault/timeout behavior?
- Is the gauge informational only or part of a safety-relevant instrument cluster?
- What visual reference, display dimensions and acceptance thresholds must be met?
- Who owns integration, test hardware, licensing, safety review and final approval?

12. Recommendation and handover statement

Submit the current work as a software UI proof of concept, not as a vehicle-ready instrument cluster. Request a short technical review with the OEM display, embedded/network and safety teams. After receiving an approved signal specification and target hardware, implement and validate the vehicle data adapter on a bench before any vehicle-level testing.

Proposed handover wording: "We present a Qt 6/C++/QML tachometer prototype demonstrating the user interface and a simulated RPM data path. Integration with a vehicle ECU requires an approved network signal specification, target hardware definition, transport implementation, validation and OEM safety/security acceptance. No production-readiness claim is made at this stage."